Business 41204

Machine Learning

Notes 2 – More Supervised Learning

Christian Hansen

University of Chicago
Booth School of Business

Damian Kozbur

University of Zurich
Economics

Outline: (??)

1. Classification
2. Extending Trees: Random Forests and Boosted Trees
3. Interpreting “Black Box” Learners
4. Stacking and AutoML

1. Classification

Classification

Classification = jargon for predicting a categorical outcome

- Predict default (yes/no)
- Predict churn (yes/no)
- Predict animal type in image (cat/dog/…/other)

Same basic concepts apply to classification and regression

- validation exercises to choose good performing models
- standard performance metrics for validation differ from standard performance metrics for regression
  - Regression metrics still make sense though

Our examples in this set of notes will be for classification.

Logistic Regression

The baseline model for classification is logistic regression

Logistic regression fits a model of the form

$$\hat{p} = \Pr(\widehat{Y = 1}|X) = \sigma\left(b_0 + b_1X_1 + \cdots + b_pX_p\right)$$

$$\sigma(u) = \frac{1}{1 + e^{-u}} = \frac{e^u}{1 + e^u}$$

- Appropriate for binary Y. Models the probability that Y belongs to the "1" class
  - Typically, predict $Y = 1$ if $\hat{p} > .5$
  - Can use other decision threshold – predict $Y = 1$ if $\hat{p} >$ decision threshold
- More complicated than linear regression but still "interpretable"
- Generalizations for settings with multiple classes (e.g. multinomial logistic regression)
- Same conversation about features and transformations from linear regression applies
- Can also use penalization as in linear models

Classification Evaluation

Typically MSE, MAE not used for evaluating classification performance

- Can be used though and do make sense

Baseline Evaluation Measures:

- Accuracy – fraction of time prediction is correct
  - Extremely simple and intuitive
  - Directly targets object that is often of interest
  - Coarse – not uncommon that many prediction rules give same accuracy
- Cross-entropy (or log) loss
  - Intuitively looking at how well model probabilities line up with data
  - Numerically - not terribly meaningful to most people (hard to interpret)
  - Less “coarse” than accuracy as it focuses on probabilities rather than the discrete predictions
  - Logistic regression based on this loss

Classification Evaluation

There are other common measures that look more specifically at true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN)

Further Evaluation Measures:

- Precision = TP/(TP + FP) = TP/(total positive predictions)
  - high when there are few FP
  - care about this when FP are costly (e.g. spam detection, fraud detection)
- Specificity = TN/(TN + FP) = TN/(total negatives in data)
  - High when there are few FP
  - 1 – Specificity = false positive rate
- Recall (aka sensitivity, true positive rate) = TP/(TP + FN) = TP/(total positives in data)
  - high when there are few FN
  - care about this when FN are costly (e.g. medical screening)
- F1 = 2TP/(2TP + FP + FN)
  - aims to “balance” FP and FN

Customer Churn

Let's look at using classification by trying to predict customer churn

- i.e. try to predict whether customer will cease doing business during some period
- Why might we be interested in predicting churn?
- Do we think there are equal costs to false positives and negatives? How might this influence our evaluation measure?

Data:

- n = 1000 customers that started period as active customers
- p = 7 raw features

Code for Flower Shop Churn Example

Code for Flower Shop Churn Example

Churn Data

Here are a few rows of our dataset:

customer_id	churn	tenure	monthly_charges	total_charges	contract	payment_method	feedback	topic
CUST001	True	12	30.00	360.0	one year	credit card	Love the variety of flowers!	bouquet preferences
CUST002	True	4	35.50	142.0	month-to-month	electronic check	Delivery was often late	delivery issues
CUST003	False	20	29.99	599.8	two year	bank transfer	Great service and quality	general feedback
CUST004	True	2	32.00	64.0	month-to-month	mailed check	Not enough variety	bouquet preferences
CUST005	True	15	28.50	427.5	one year	credit card	Beautiful arrangements every time	general feedback

The outcome variable is the binary variable `churn`.

We have

- "continuous" features – `tenure, monthly_charges, total_charges`
- categorical features – `contract, payment_method, topic`
  - include as dummy variables
- text feature – `feedback`
  - include by "encoding" sentiment

Aside: Text as a Feature

- Often have natural language that we would like to use as a feature
  - Generally cannot simply introduce a dummy for each unique phrase
- Lots of potential ways to “encode” text
  - “bag of words”, n-grams
    - “high-dimensional” and may not capture actual meaning
  - read each text and hand code “semantic content”
    - e.g. could read each of the 1000 feedback entries and code as “positive” or “negative”
- Advances in NLP/LLMs let us automate the extraction of content
  - Find an existing NLP model
  - Have NLP model process text
  - Use features of the NLP model to represent the information in the text
  - Still many choices – we’ll talk about some (later)

Code for Flower Shop Churn Example

Sentiment in Churn Data

My intuition suggests the chief information from `feedback` about churn is the feedback's sentiment – positive or negative

Hugging Face is a great resource that has many pre-trained models for language and image processing tasks – including sentiment analysis

Going to use "distilbert-base-uncased-finetuned-sentiment-amazon"

- Yes, the name is essentially meaningless
- Basic idea – starts from a general language model (BERT, a couple generations old but akin to ChatGPT)
- "Fine tunes" the model using data from Amazon reviews that include the "sentiment" of the reviews

- Model outputs a sentiment summary (positive/negative) and "probability" that the classification is correct for a given text input
- Can use the probability as a continuous feature capturing how strongly positive/negative the review is
- Can use the binary summary positive/negative as a categorical feature
- Don't have to read and hand code every feedback entry
- Example of feature engineering

Code for Flower Shop Churn Example

Sentiment in Churn Data

Let's see what we get with some "toy" phrases:

Phrase	Sentiment	Score
I love this!	positive	0.991
I hate this!	negative	0.982
Things are ok.	positive	0.567
Exceptional quality every month	positive	0.993

Phrase from the data

Model's "estimate" of the probability that the assigned Sentiment is correct.

Overall, seems like we're capturing the sentiment of these phrases.

Note that there is no "neutral" output.

For our model, we are going to define a new variable, `polarity`, that is equal to `score` if `sentiment` is positive and 1-`score` if `sentiment` is negative.

Code for Flower Shop Churn Example

Sentiment in Churn Data

Sentiment:

- 485 positive
- 481 negative
- 34 empty

Polarity:

- mean = 0.53
- max = 0.998
- min = 0.001

5 lowest polarity phrases

feedback	polarity
Not worth the price	0.001395
Unresponsive customer service	0.001894
Unsatisfactory delivery service	0.002256
Not worth the monthly cost	0.001602
Not worth the monthly fee	0.002463

5 highest polarity phrases

feedback	polarity
Excellent variety and freshness	0.998001
Always on time and fresh	0.997889
Always fresh and beautiful	0.997499
Flowers always arrive fresh and beautiful	0.997278
Lovely and fresh bouquets	0.997233

At least from these summaries, looks “reasonable.”

Code for Flower Shop Churn Example

Baseline Churn

In the churn dataset, we have

- 509 observations with `churn` = 1 (and 491 with `churn` = 0)
  - Hope these are not representative of actual customer turnover!

Data have likely been purposely "balanced" to deal with class imbalance

- If a class is particularly rare, can be hard to build a model that distinguishes between classes
- E.g. if a class occurs 99% of the time, a model that simply ignores features and always predicts that class will have 99% accuracy
- With extreme imbalance, data are often artificially balanced to make differentiating classes on the basis of features easier
- Fine, but need to be careful when interpreting, e.g., predicted probabilities

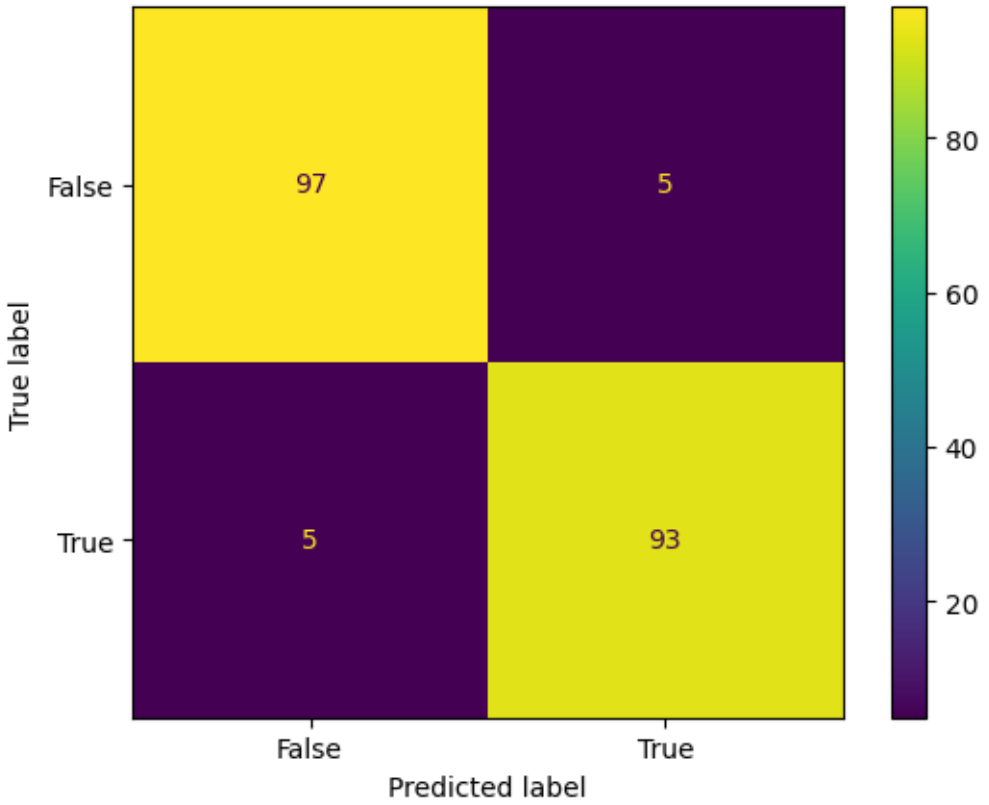
Logistic Regression Performance

Code for Flower Shop Churn Example

Let's see how a logistic regression classifier actually performs on our data.

- Estimate the model with 80% of the data
  - 411/800 have `churn` = 1
- Evaluate on the held out 20%.
  - 98/200 have `churn` = 1
  - Baseline accuracy = 0.49 = 98/200

Confusion Matrix from logistic regression in validation data:



The confusion matrix shows prediction results by displaying

TP (93), FP (5),
TN (97), FN (5).

Accuracy:
= 0.95 = (93+97)/200

correct predictions

total predictions

Code for Flower Shop Churn Example

Other Common Accuracy Statistics

Given the TP, FP, TN, FN from the confusion matrix, can compute precision, specificity, recall, and f1.

	True
precision	0.94898
recall	0.94898
specificity	0.95098
f1-score	0.94898

Precision = TP/(TP + FP) = 93/(93+5) = .94898

Recall = TP/(TP + FN) = 93/(93+5) = .94898

Specificity = TN/(TN + FP) = .95098

f1 = 2TP/(2TP + FP + FN) = 2*93/(2*93+5+5) = .94898

It's an "accident" that precision, recall, and f1 all happen to line up.

ROC and AUC

Code for Flower Shop Churn Example

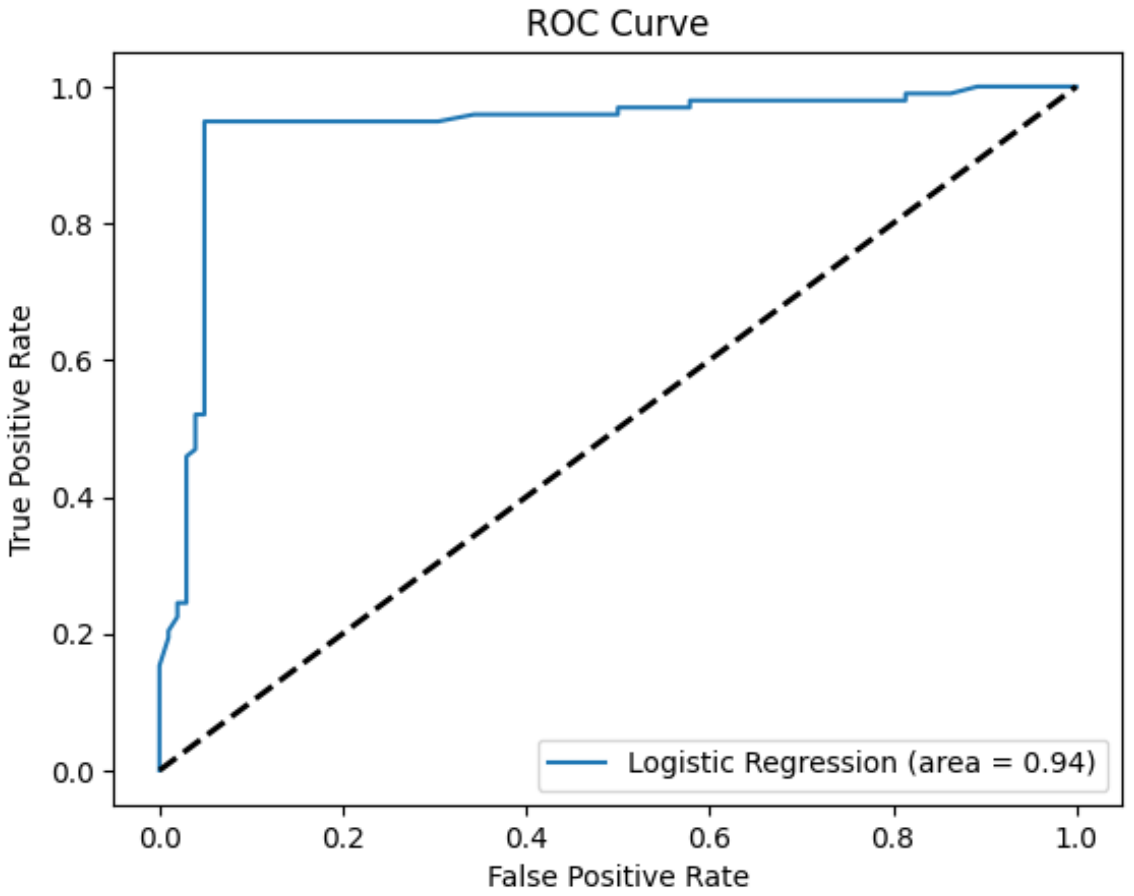
ROC is another popular summary of classification quality

ROC plots

$$TPR = TP/(TP + FN) \text{ vs } FPR = FP/(FP + TN)$$

by varying decision threshold between 1 and 0

- when decision threshold is 1, all classified as 0 so no FP or TP
- as threshold decreases, start getting FP and TP
- random guessing gives equal FP and TP (on average)
- perfect classification would be all TP and no FP



AUC – area under the ROC curve

- best possible is 1
- random guessing is .5

[Code for Flower Shop Churn Example]

Cumulative Gain

Thought experiment:

- Contact X% of customers/population
- See how many positives you get

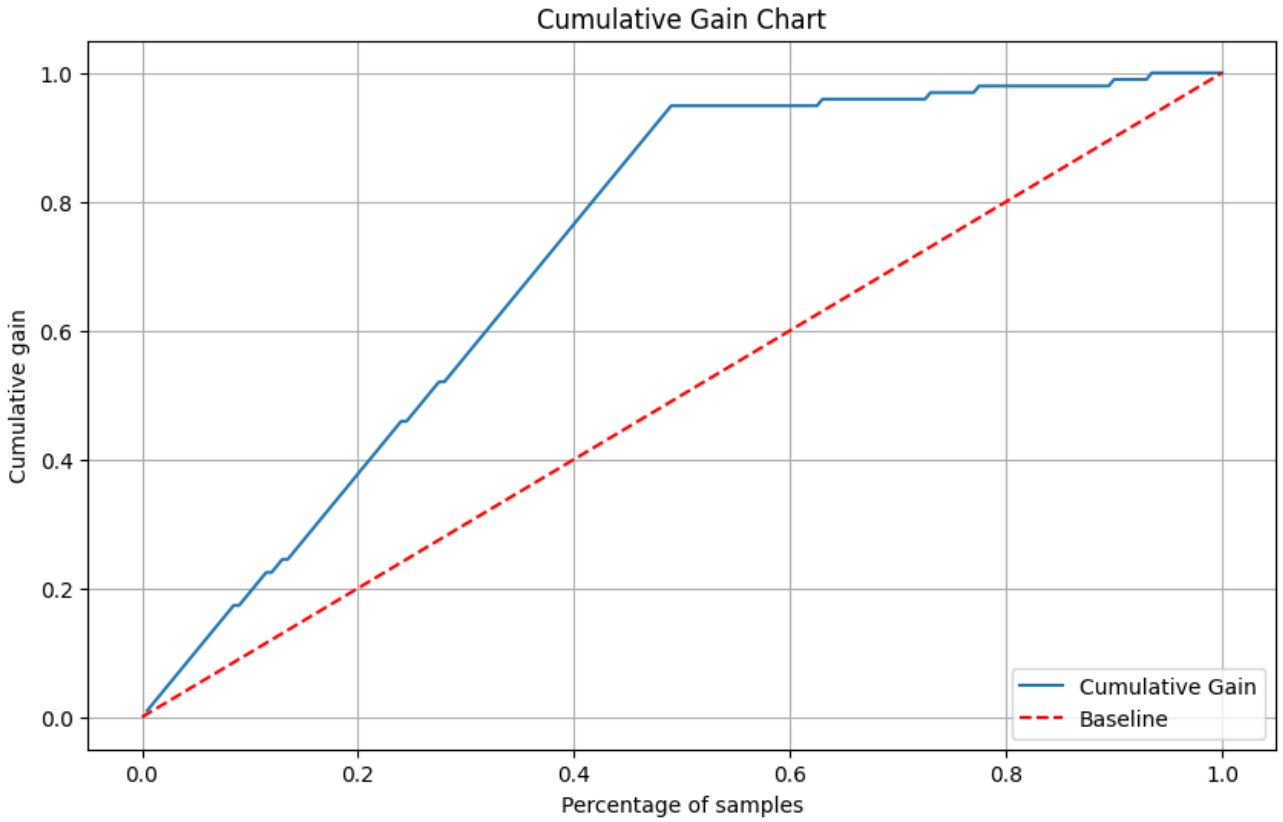
X-axis – fraction of sample “contacted”
Y-axis – fraction of true positives “reached”

Baseline – contact at random

- e.g. would expect to get 10% of total positives if contact 10% at random

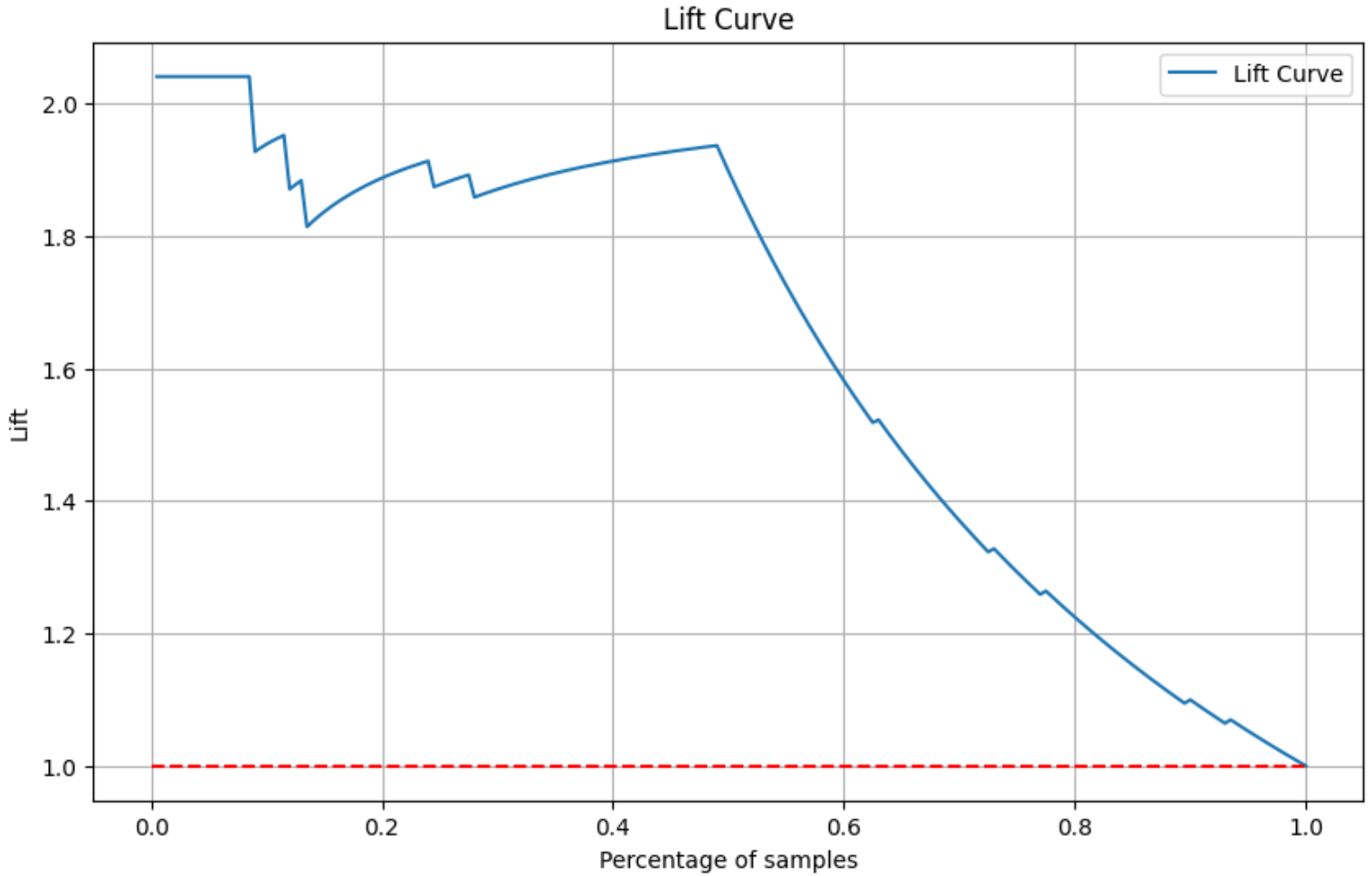
Gain – rank observations according to predicted probability, “contact” in order of ranking

- shows gain of using predictive model



[Code for Flower Shop Churn Example]

Lift



Lift: Simply

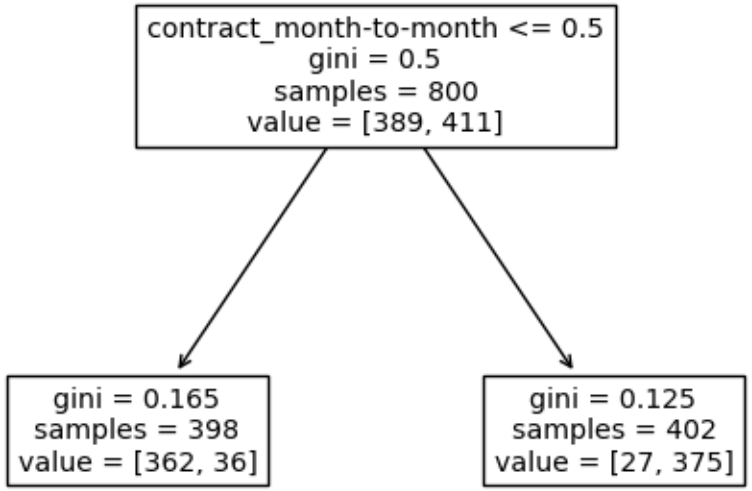
$$\frac{\text{Gain}}{\text{Baseline}}$$

Shows how much more likely to reach positive class by using model than by "contacting" at random

[Code for Flower Shop Churn Example]

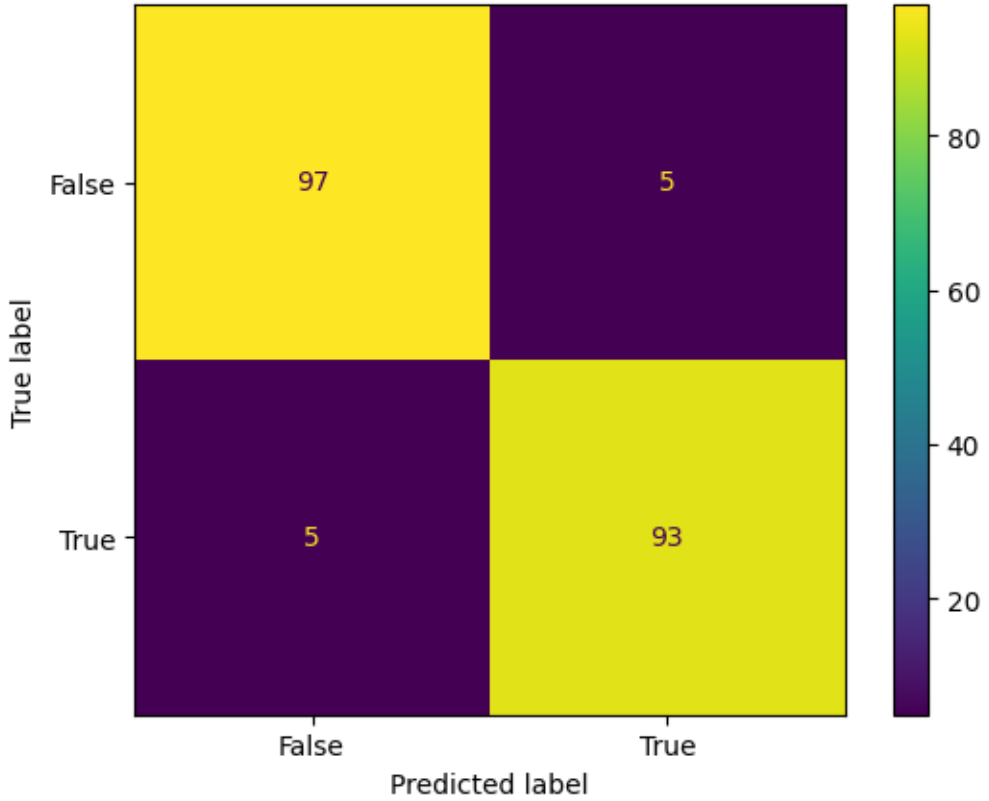
Classification Tree

For fun, let's try a tree as well.



CV minimizing tree

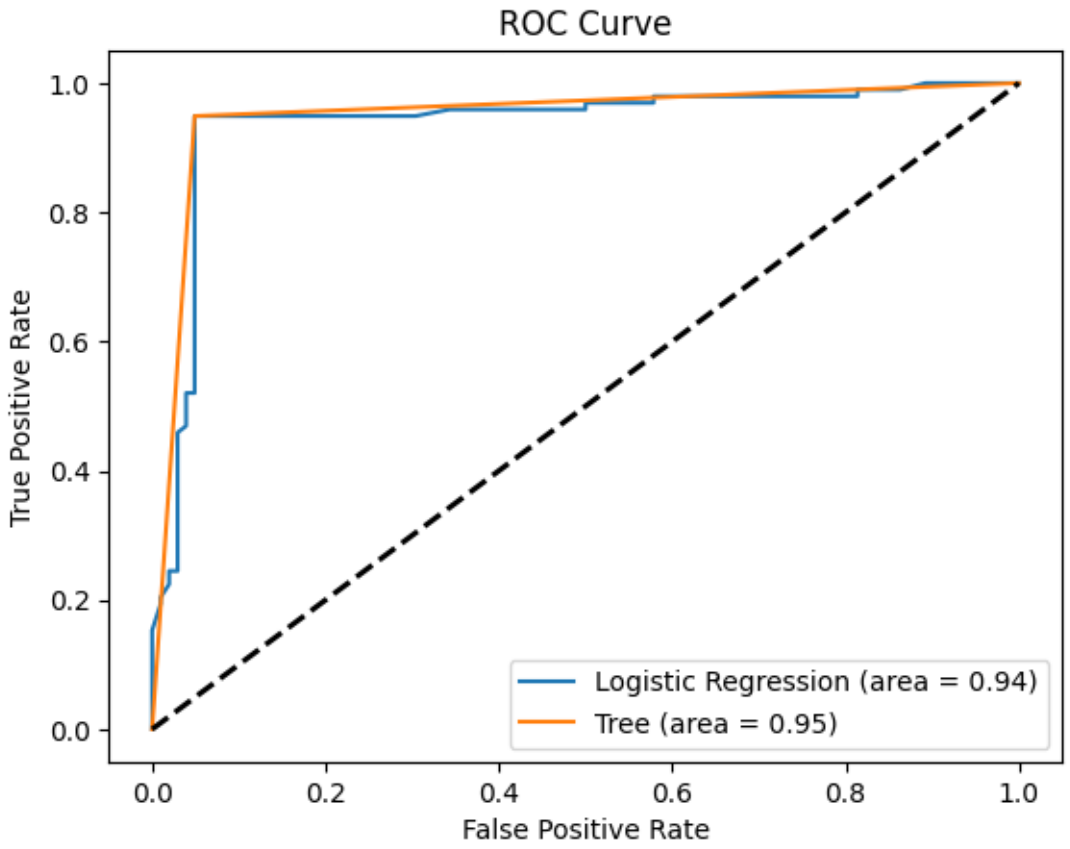
- *pretty simple*!

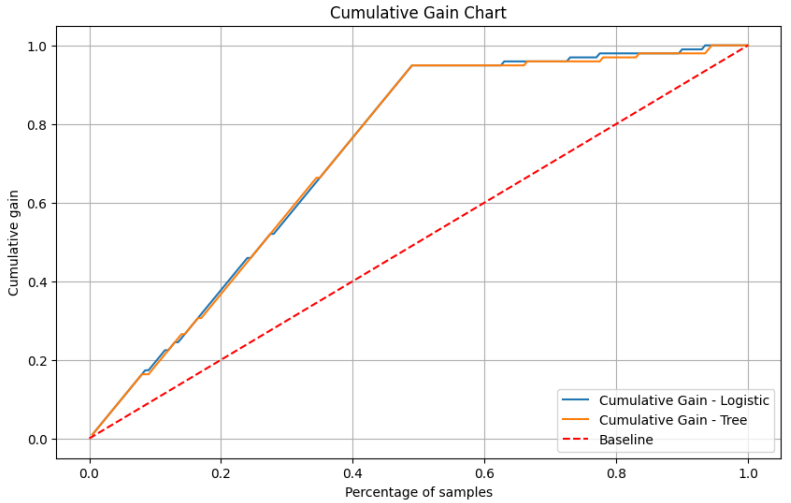


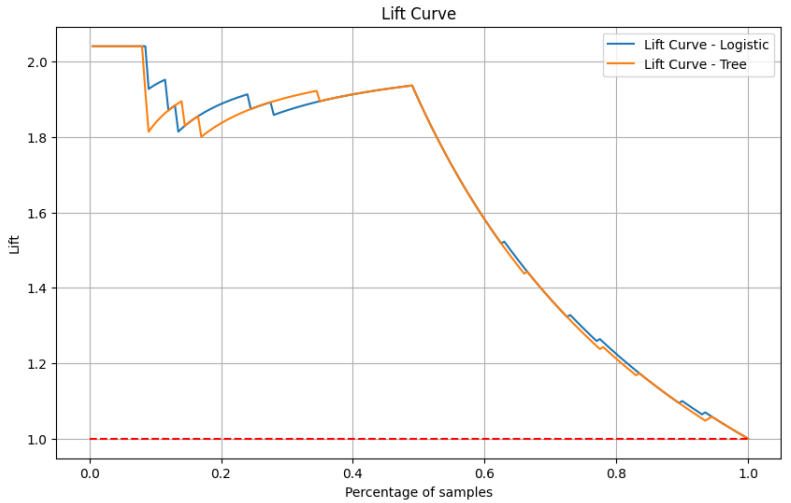
Confusion matrix for tree – identical to logistic

Tree – ROC, AUC, Lift

[Code for Flower Shop Churn Example]







More on Evaluation

Why do we care about predicting churn in the first place?

- Presumably so we can do something

Maybe we are interested in targeting individual's predicted to churn with an incentive to stay.

A simple model:

$$V = S(V_S - cost_S) + C(V_C - cost_C)$$

- V = value of a customer
- S = Stay; V_S = value of customer if Stay, $cost_S$ = cost of targeting if Stay
- C = Churn; V_C = value of customer if Churn, $cost_C$ = cost of targeting if Churn

Expected Value of Targeting

Can evaluate expected value of individuals (with characteristics X) with and without targeting (T):

$$E[V|T,X] = \Pr(C=0|T,X)(E[V_S|T,X] - E[cost_S|T,X]) + \Pr(C=1|T,X)(E[V_C|T,X] - E[cost_C|T,X])$$

Some simplifying assumptions:

- $E[V_C|T,X] = 0$
- $E[V_S|T,X] = E[V_S|X]$
- $E[cost_S|T=1,X] = E[cost_C|T=1,X] = cost$
- $0 = cost_S|T=0 = cost_C|T=0$

What do these assumptions mean?

Are they realistic?

Would it be hard to relax them?

$$\Rightarrow E[V|T,X] = \Pr(C=0|T,X)(E[V_S|X] - T*cost) - \Pr(C=1|T,X)*T*cost$$
$$= \Pr(C=0|T,X)E[V_S|X] - T*cost$$

Expected Value of Targeting

So, expected value of targeting:

$$E[V|T=1,X] - E[V|T=0,X] = \big(\Pr(C=0|T=1,X) - \Pr(C=0|T=0,X)\big)E[V_S|X] - cost$$
$$= (\Pr(C=1|T=0,X) - \Pr(C=1|T=1,X))E[V_S|X] - cost$$

Intuition: Target people with high value (high $E[V_S|X]$) weighted by how much targeting impacts the probability that they stay. I.e.

- don't bother with low value people (no matter how responsive to the targeting they are)
- don't bother with people that don't respond to the targeting (no matter how valuable they are)

Model gives us estimate for $\Pr(C=1|T=0,X)$ (Assuming a new targeting campaign)

- Where do we get information on $\Pr(C=1|T=1,X)$, $E[V_S|X]$?
  - Historical data might give us a way to estimate on $E[V_S|X]$
  - Ideally we'd run a trial (e.g. an A/B test) to see how the targeting impacts the probability of staying (and spending/value)
  - Otherwise, we can make things up

If we did actually implement the targeted incentive, how would we decide if it was effective?

2. Extending Trees: Random Forests and Boosted Trees

“Modern” Learners

We’ve reviewed old fashioned learners:

- regression (linear and logistic), trees
- often work well – especially in “classical” settings
- interpretable(?)

Have some drawbacks:

- regression requires many choices
- basic trees are “crude” (though there are lots of variants)

“Modern” learners aim to automate feature construction (like trees) while providing higher quality approximations than trees

- random forests
- boosted trees
- (deep) neural networks (LATER)

Random Forest

Random forest:

- based on a large number of trees (a forest)
- each tree is trained using random data (random)
  - Two main approaches:
    - Take a random subsample of the available training data
    - Take a random sample ***with replacement*** from the training data
  - Typically also take a random subset of the available predictors
- Special case of a powerful idea called "bagging" due to L. Breiman (1996) "Bagging predictors" *Machine Learning* 26, 123-140.

In math:

- Let b denote the b^{th} randomly generated set of data
- Let $\widehat{tree}_b(x)$ be the tree tree prediction rule learned using data set b
- The random forest prediction rule is $\widehat{rf}(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{tree}_b(x)$ where B is the total number of estimated trees

Some “Intuition” for Random Forests

Why should random forests work?

- Remember, trying to make “bias/variance” tradeoff
  - What would happen if all the trees were unbiased and independent?
  - Easy to fit “low-bias” trees – Use lots of leaves
- Will trees fit on different data produce the same splits?
  - What happens if you average step functions with steps at different points?

In principle, lots of tuning choices for random forests

- tend to work well “out-of-the-box” though
- part of the sales pitch is relative insensitivity to tree specific choices
  - use a large B
  - use “big” trees

Still judging quality based on out-of-sample prediction performance

Code for Bank Deposit Example

Bank Example

Goal: Predict whether a person subscribes to a bank term deposit in response to a telemarketing campaign

Data:

- n = 41188 customers
- p = 19 predictors
  - include demographic and macroeconomic variables
- S. Moro, P. Cortez and P. Rita (2014) “A Data-Driven Approach to Predict the Success of Bank Telemarketing. Decision Support Systems,” *Decision Support Systems* 62, 22-31.

- Use 80% for training, 20% validation

Code for Bank Deposit Example

Bank: Baseline Models

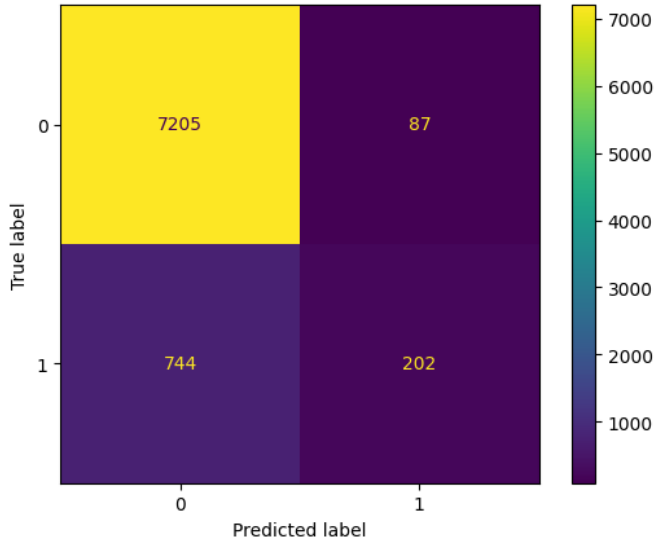
Two baseline models:

1. Logistic regression with raw predictors
   - Accuracy = 0.899
2. Constant (i.e. ignore predictors)
   - Accuracy = 0.885

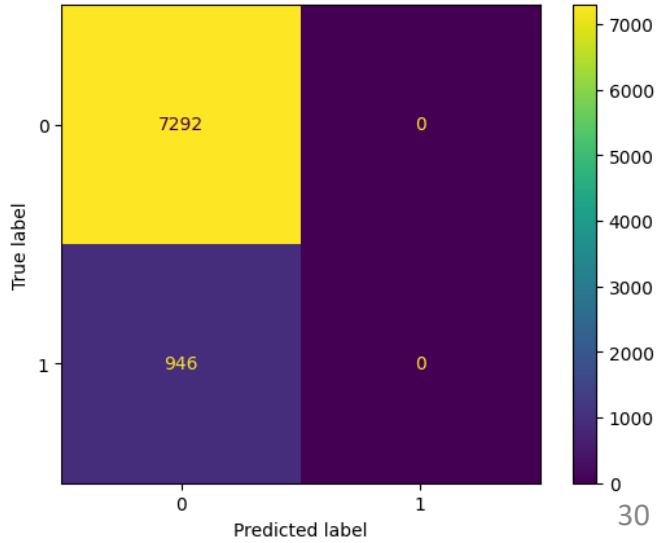
Ignoring all predictors is almost as accurate as logistic regression. What do we care about?

Why is ignoring the predictors so accurate?

Logistic regression confusion matrix



Constant prediction confusion matrix

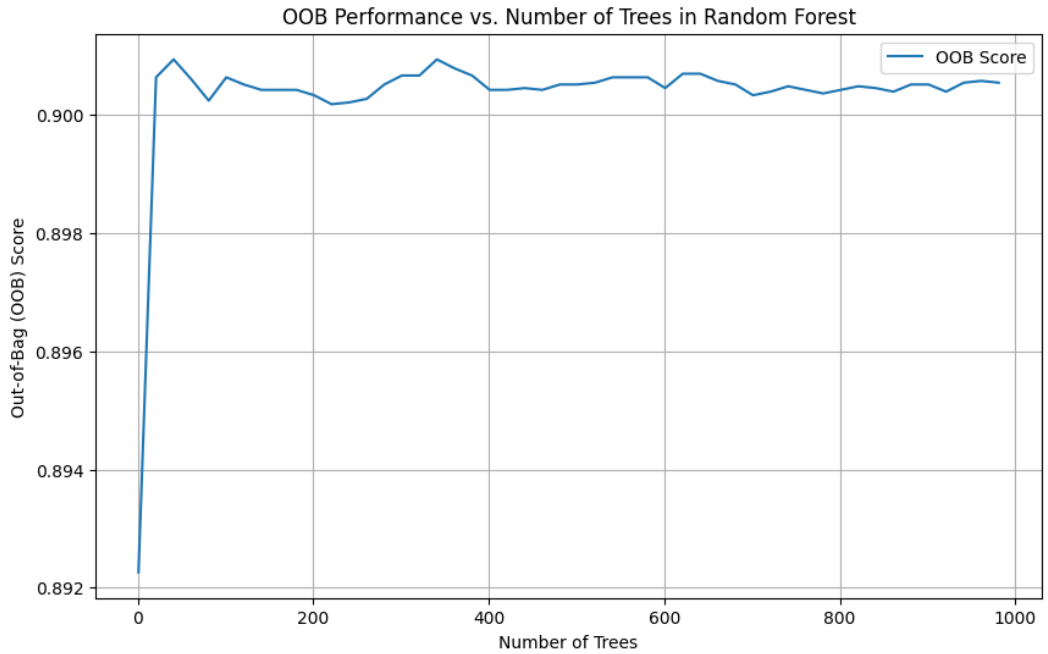


Code for Bank Deposit Example

Bank: Random Forest

Let's try random forests out in this example.

- Use cross-validation to choose *B* and minimum number of observation per leaf in the tree
  - Choose *B* = 1000 and minimum observations per leaf = 15



Plot of "out-of-bag" accuracy against *B* with minimum 15 observations per leaf.

Relatively stable after 50 or so trees.

Could probably have saved some computation time by considering fewer trees.

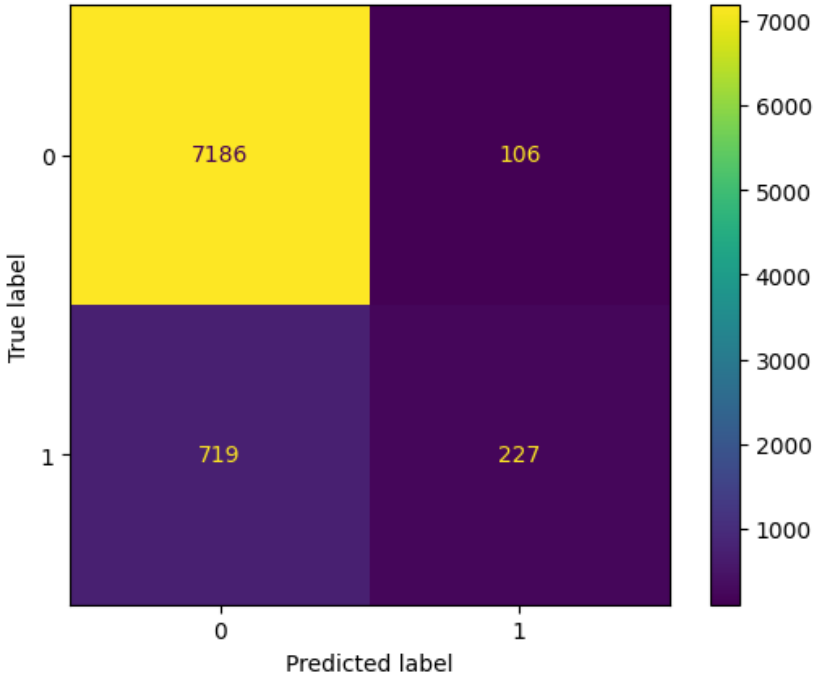
A fun feature of random forests is that all of the observations not included in the resampled data are "out-of-sample" (out-of-bag) and can be used for validation

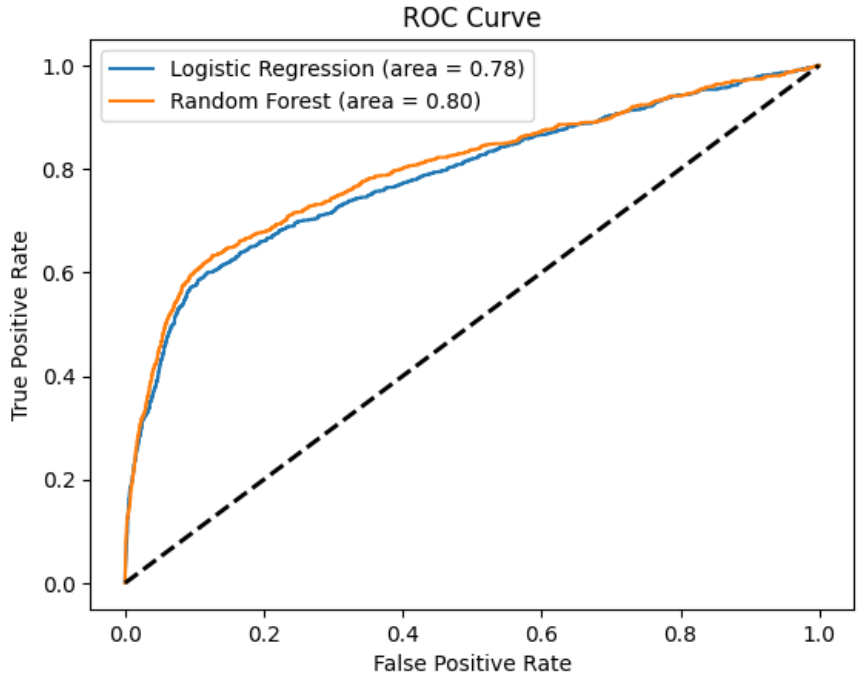
Code for Bank Deposit Example

Bank: Random Forest

Random forest accuracy: 0.900

- Recall constant gave 0.885 and logistic regression 0.899





What would ROC for the constant model look like?

Looks a little better than logistic regression.

Boosting

Boosting:

- basic idea is to improve model fit in small increments by *recursive fitting*
  - Fit a simple model and obtain prediction errors
  - Fit a new simple model to the prediction errors and construct new prediction errors
  - Repeat …
  - Final prediction rule is sum of all prediction rules
- each step must improve fit (within training data) relative to previous step
  - overfit if too many steps taken – typically do (cross-)validation to choose number of steps
- popular with tree methods
- R. E. Schapire. The strength of weak learnability. Machine Learning, 5:197–227, 1990.

Boosted Trees

Basic boosting algorithm for trees with data $\{y_i, x_i\}_{i=1}^n$:

1. Initialize *residuals* (prediction errors) as $r_{i,1} = y_i$
2. For b = 1,…,B
   1. Fit tree based prediction rule $\widehat{tree}_b(x)$ using data $\{r_{i,b}, x_i\}_{i=1}^n$
      - Should be a simple tree. E.g. a tree of depth d for d small
   2. Update the residuals to $r_{i,b+1} = r_{i,b} - \lambda \widehat{tree}_b(x_i)$
      - λ is called the learning rate and plays an important role. Usually want $\lambda \ll 1$. (We'll see learning rates again with deep neural networks)
3. Construct the final prediction rule as $\widehat{boost}(x) = \sum_{b=1}^B \lambda \widehat{tree}_b(x)$

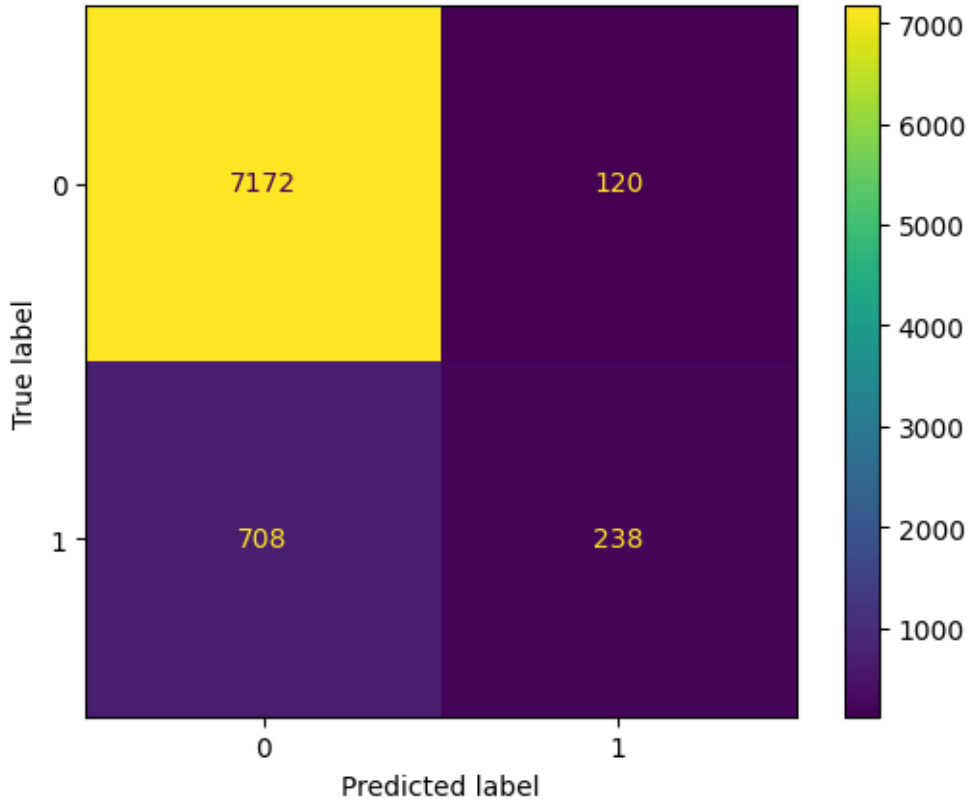
Tuning parameters - λ, B, d (or other tree parameters) chosen by (cross-)validation

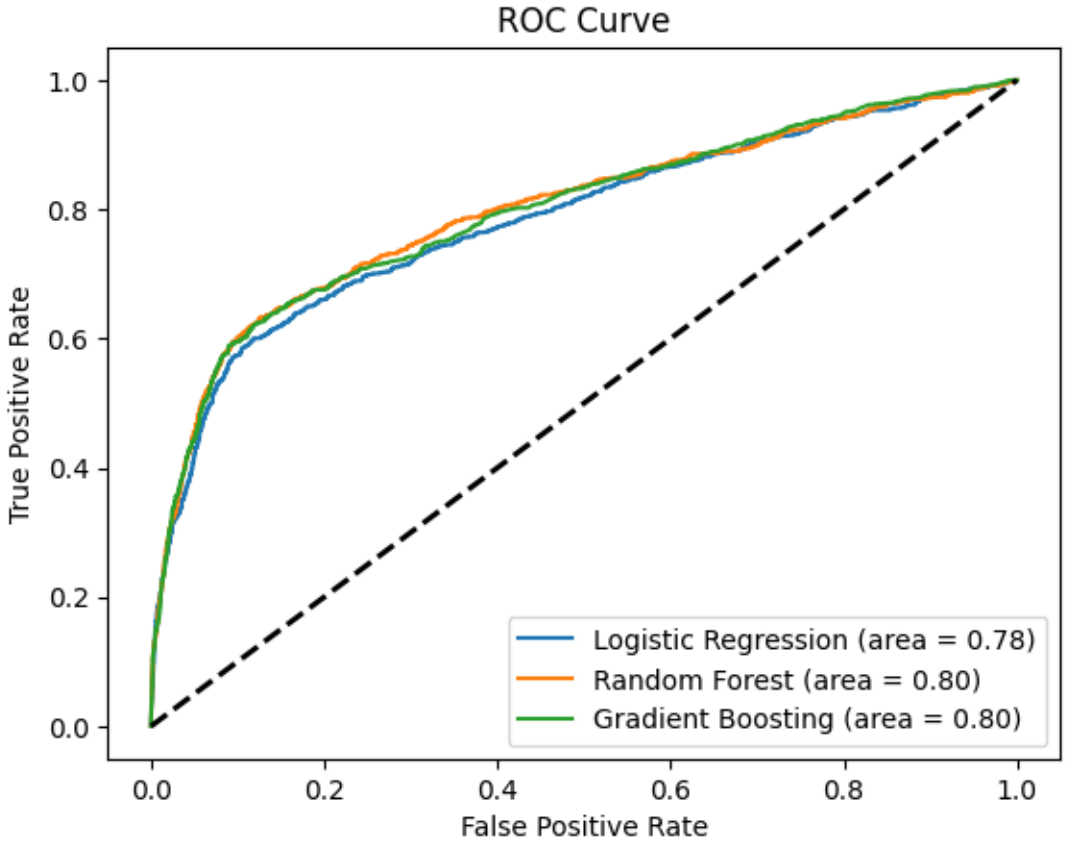
Code for Bank Deposit Example

Bank: Boosted Trees

Boosted tree accuracy: 0.899

- Recall constant gave 0.885, logistic regression 0.899, random forest 0.900





Early Stopping

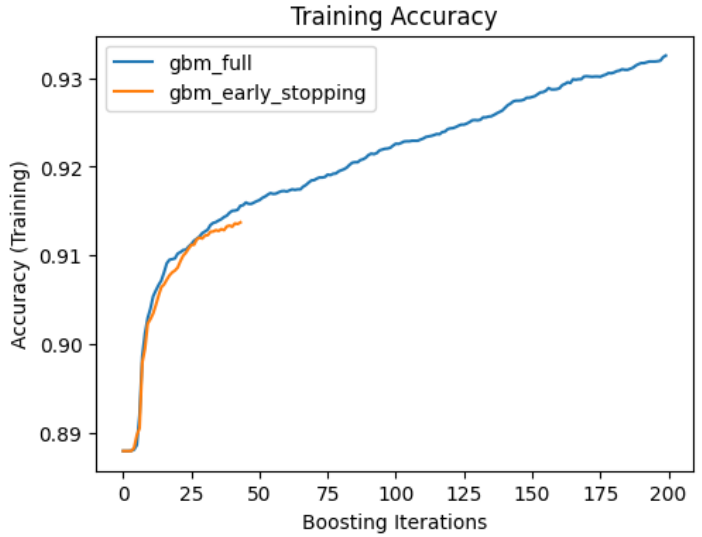
Many learners (including boosting)

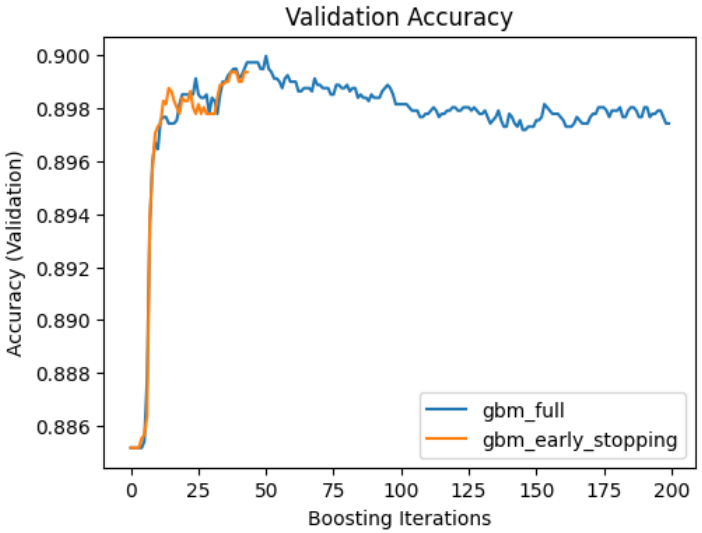
- are updated in a step by step fashion
- are such that beyond a certain number of steps, training fit will continue improving but out-of-sample performance is likely to degrade
- instead of training a fixed number of step, can monitor validation performance and stop when improvement levels off/worsens

= ***Early Stopping***

Can save computation time AND improve prediction performance!

Code for Bank Deposit Example



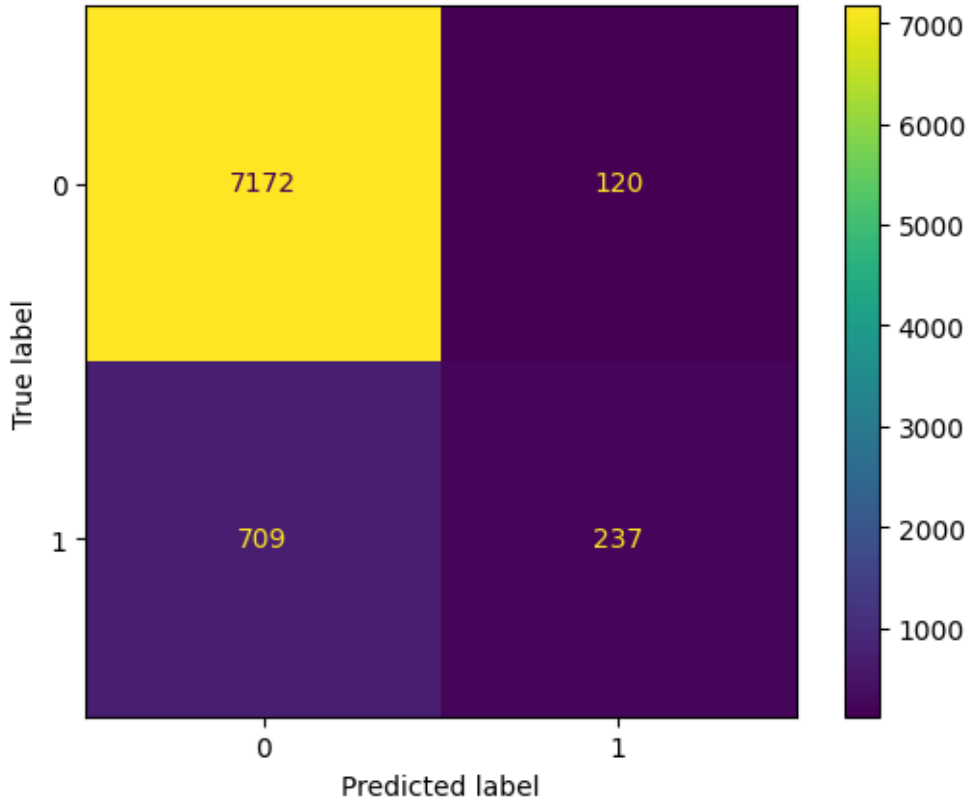


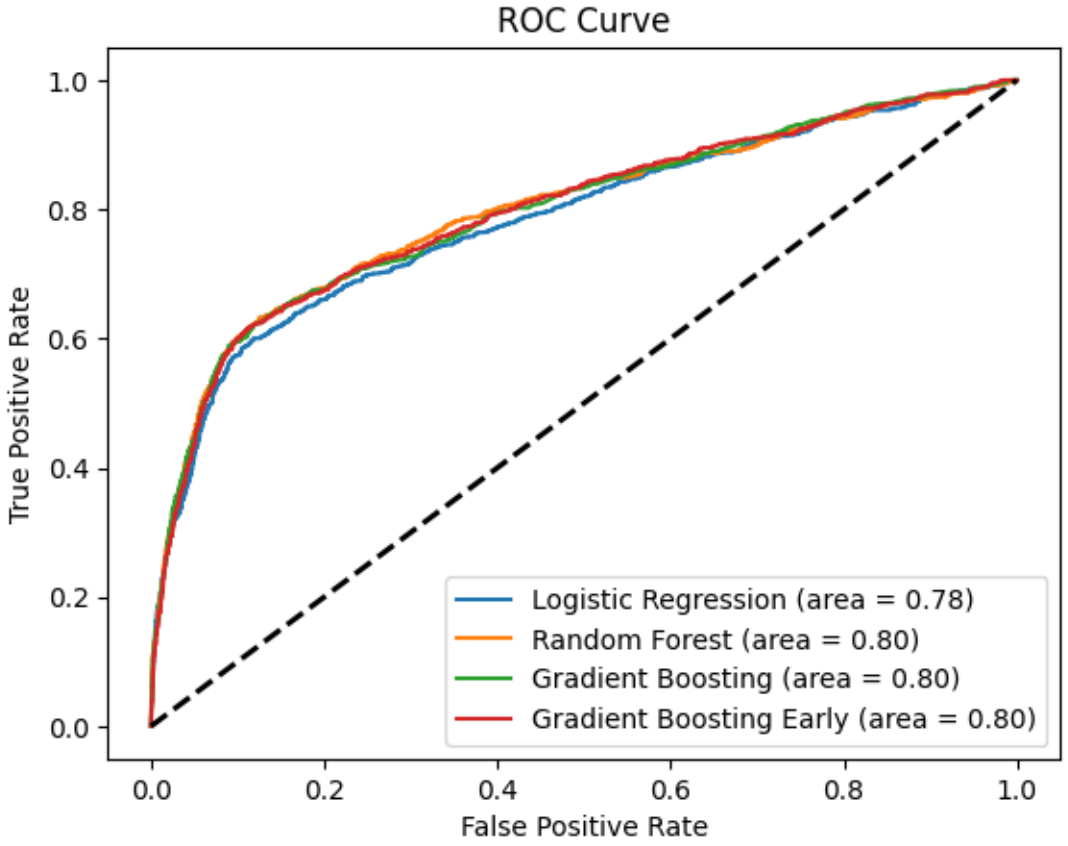
Code for Bank Deposit Example

Bank: Boosted Trees with Early Stopping

Boosted tree accuracy: 0.899

- Recall constant gave 0.885, logistic regression 0.899, random forest 0.900





“Expected Profit”

Might want to use “economic” criterion to choose model

Things we might think about in stylized setting for determining the lifetime value of a contact resulting in a new account:

1. Initial Deposit and Balance Maintenance over time
   - Capital the bank can use for loans and investments
2. Interest and Fee Income
   - Interest Revenue generated from the customer's account balance
   - Fee Income from the customer such as account maintenance fees, overdraft fees, and transaction fees
3. Cross-Selling Opportunities
4. Customer Retention and Longevity
   - Average Customer Lifespan/Churn Rate
5. Cost of Acquisition and Servicing
   - Costs associated with acquiring a new customer, including marketing and sales expenses.
   - Cost of servicing the account, including customer service and account management.
6. Discount Rate

We’re not going to consider most of these in our little example.

Stylized Bank Deposit Revenue

Let's make up some numbers for use in our example.

- Suppose cost C = 100 of contacting an individual
- Suppose customers who open accounts are all the same
  - Initial deposit of 1000, 2000 average maintained balance
  - Assessed fees 50/year
  - Take out credit card that generates 100/year in fees and interest
  - Stay with bank for 5 years
- Suppose the bank
  - Earns 5%/year on loans made from deposits
  - Incurs 100 in initial costs and 20/year for account maintenance, setup, …
  - Has a discount rate of 5%

Net customer revenue from account opener is then 130 in year 1 and 230 in years 2-5.

- Gives LTV of customer of ≈ 900

Code for Bank Deposit Example

Bank Deposit Benefit

We can then calculate "expected" benefit of contacting someone we predict will open account:

$$PR(TP)(900 - 100) + PR(FP)(-100)$$

where we use our prediction models to fill in estimates from the true positive and false positive rates.

Model	Estimated "Expected" Profit
Constant	0.0
Logistic	18.56
Random Forest	20.76
Boosted Tree	21.66
Boosted Tree (Early Stopping)	21.56

Using a rule that contacts if predicted takeup probability $\geq 50\%$

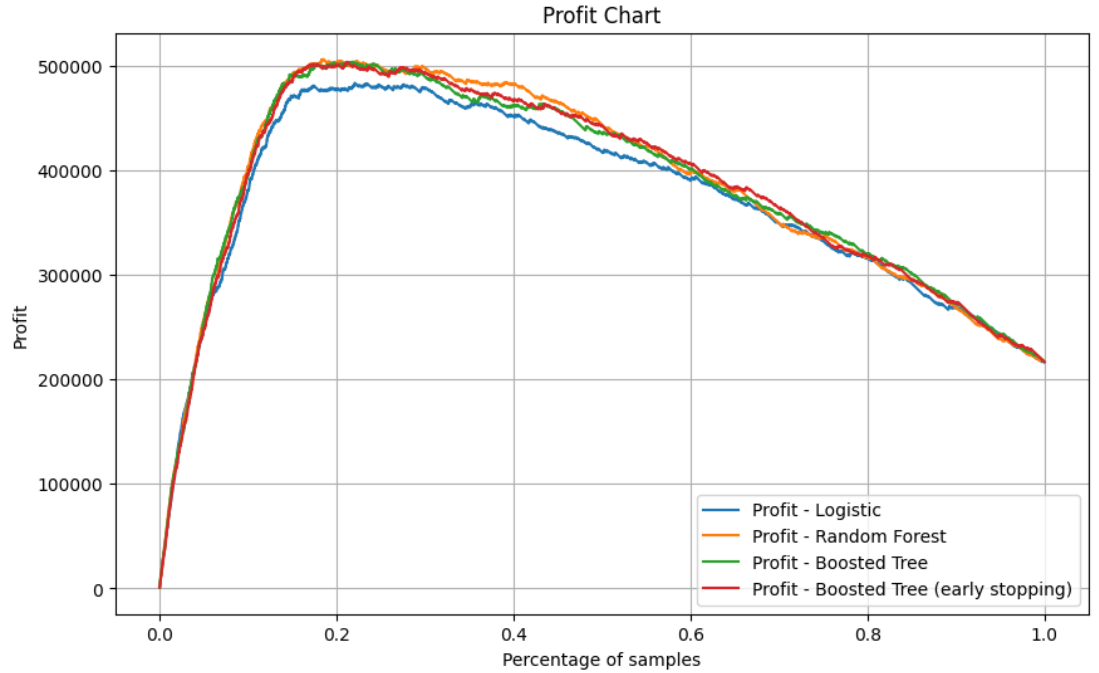
Why is this 0?

Code for Bank Deposit Example

Bank Deposit "Profit" Chart

A different exercise, akin to ROC, is to construct a "profit" chart:

1. First, order all individuals in the sample according to predicted probability of take-up.
2. Consider varying the threshold for contact (or alternatively the fraction of the sample to contact) between 0 (contact no one) and 1 (contact everyone)
3. Choose the model and threshold that gives the highest "profit." (All the true ones give a benefit of R-C, and all the true zeros give a benefit of -C.)



Model	Threshold	"Profit"	Accuracy
Random Forest	0.127	506300	0.846
Boosted Tree	0.010	502700	0.823
Boosted Tree (Early Stopping)	0.111	505100	0.844

Accuracy is lower than baselines (.885-.900)!
Do we care?

3. Interpreting “Black Box” Learners

“Black Box” Models

Random forests, boosted trees examples of “black box” prediction algorithms

- Produce forecasts, but it’s hard to see exactly what goes into prediction rule
  - Many (all?) flexible prediction procedures have this property
  - E.g. even linear models with many variables, interactions, transformations hard to think about

One sensible option is just to use such rules to make predictions and not try to (over-) interpret the results

- Good enough for many applications

Common Interpretation Devices

Sometimes one does want to dig a bit deeper into how the prediction rule works

Three commonly used interpretation devices:

- Variable importance measures
  - There are many importance measures – we'll just look at a couple
- Partial dependence plots
- SHAP values

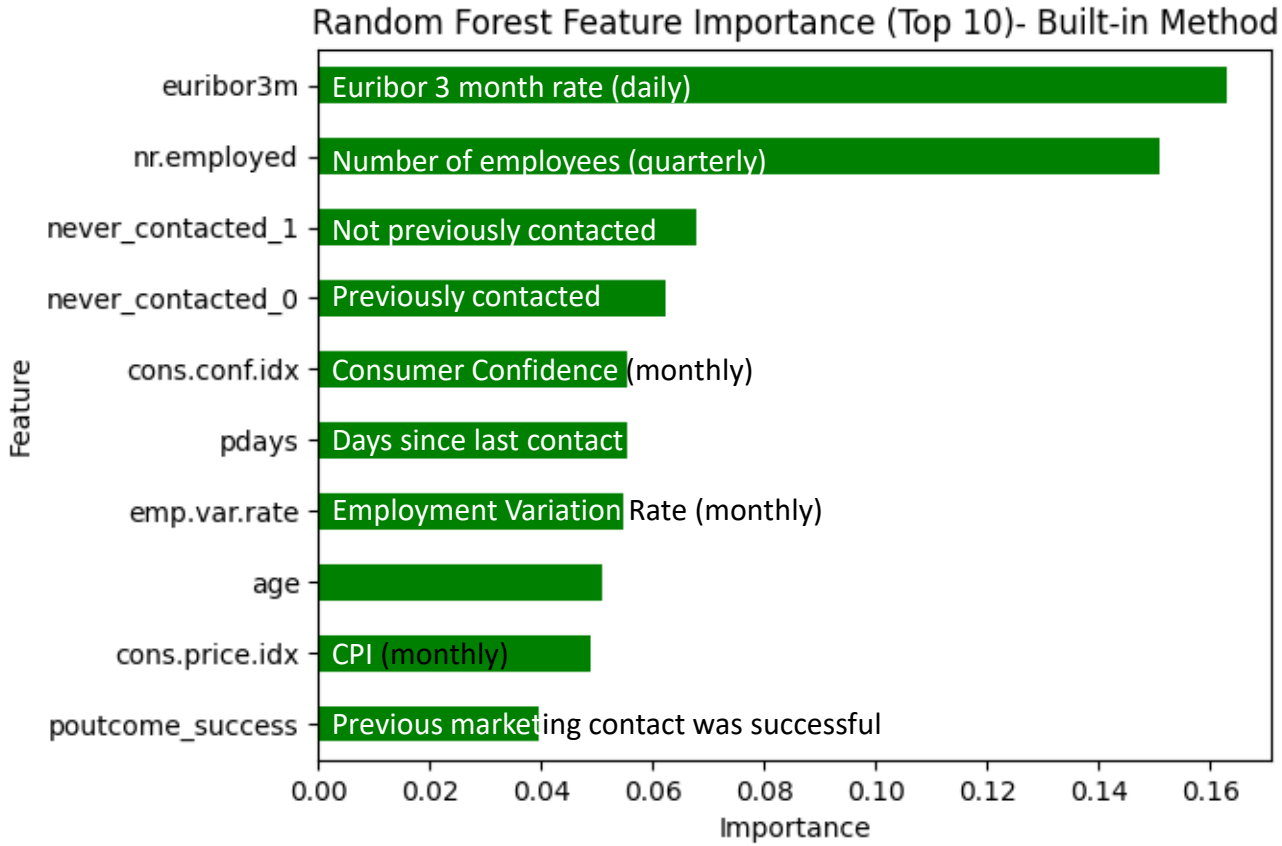
Let's look at some examples of these using the random forest model fit in the bank data.

Code for Bank Deposit Example

MDI Variable Importance

During training of tree-based methods, MDI (mean decrease in impurity) variable importance is calculated

- keeps track of variable used at each split in tree
- keeps track of improvement in fit – "decrease in impurity" at each split
- "important variables" make a large contribution to fit
- often normalized to sum to 1
- in-sample measure of importance



Code for Bank Deposit Example

Drop Column Importance

Drop column importance is a simple and robust way to measure feature importance

1. Compute prediction rule using all features and obtain performance measure (typically out-of-sample)
2. For j = 1,…,*p* (where *p* is the total number of features)
   a) Compute prediction rule using all features *excluding* feature j
   b) Obtain performance measure
3. Calculate importance of each feature by comparing performance with feature dropped to performance with all features

Advantages:

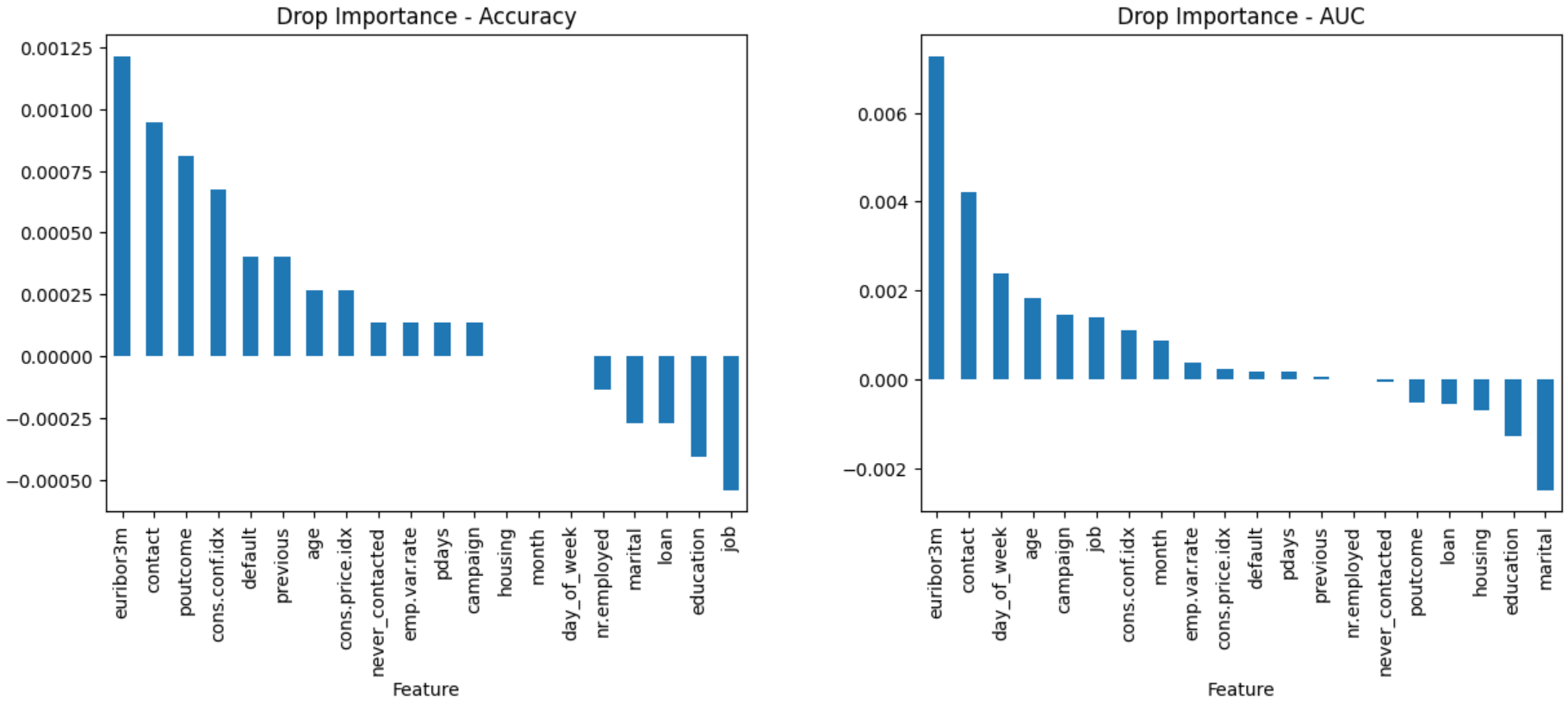
- Robust
- Can be applied to any learner
- Accounts for interactions

Disadvantages:

- Computation
- Highly correlated features

[Code for Bank Deposit Example]

Bank Example: Drop Column Importance



Drop column importance in bank example with importance based on accuracy (left) or AUC (right).

Importance calculated as $\frac{baseline\ performance - performance\ excluding\ variable}{baseline\ performance}$

Partial Dependence Plots

Any prediction rule is a mapping, $f(\cdot)$, that tells us how to translate a value of our predictor variables, x, to a predicted value for the outcome, $\hat{y}$.

- If the predictor variables were simple (e.g. there was only one predictor), we could understand $f(\cdot)$, no matter how complicated, by just plotting $f(x)$ at many values of x
- Unfortunately, not practical for complex x

Partial dependence plot takes this idea and plots a *summary* of $f(x)$ focusing on one variable at time

- Suppose you are interested in understanding how variable X_j relates to the prediction rule. Let X_{-j} denote all the other variables
- Typical partial dependence plot essentially proceeds as follows:
  1. Choose a set of values for X_j: $(\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_\mathrm{M})$
     - E.g. if X_j denoted age, you might choose values (16,17,18,…,99,100)
  2. For each value $m = 1, \ldots, M$, compute $\bar{f}_j(a_m) = \frac{1}{B}\sum_{i=1}^{B} f(X_j = a_m, X_{-j} = x_{-j,i})$
     - I.e. fix the value of variable X_j, compute the prediction with all other variables set equal to values in a dataset (training, test, or some auxiliary) for all values in that data, take the sample average
  3. Plot the result. I.e. plot $(\bar{f}_j(a_1), \ldots, \bar{f}_j(a_M))$ against $(\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_\mathrm{M})$

Code for Bank Deposit Example

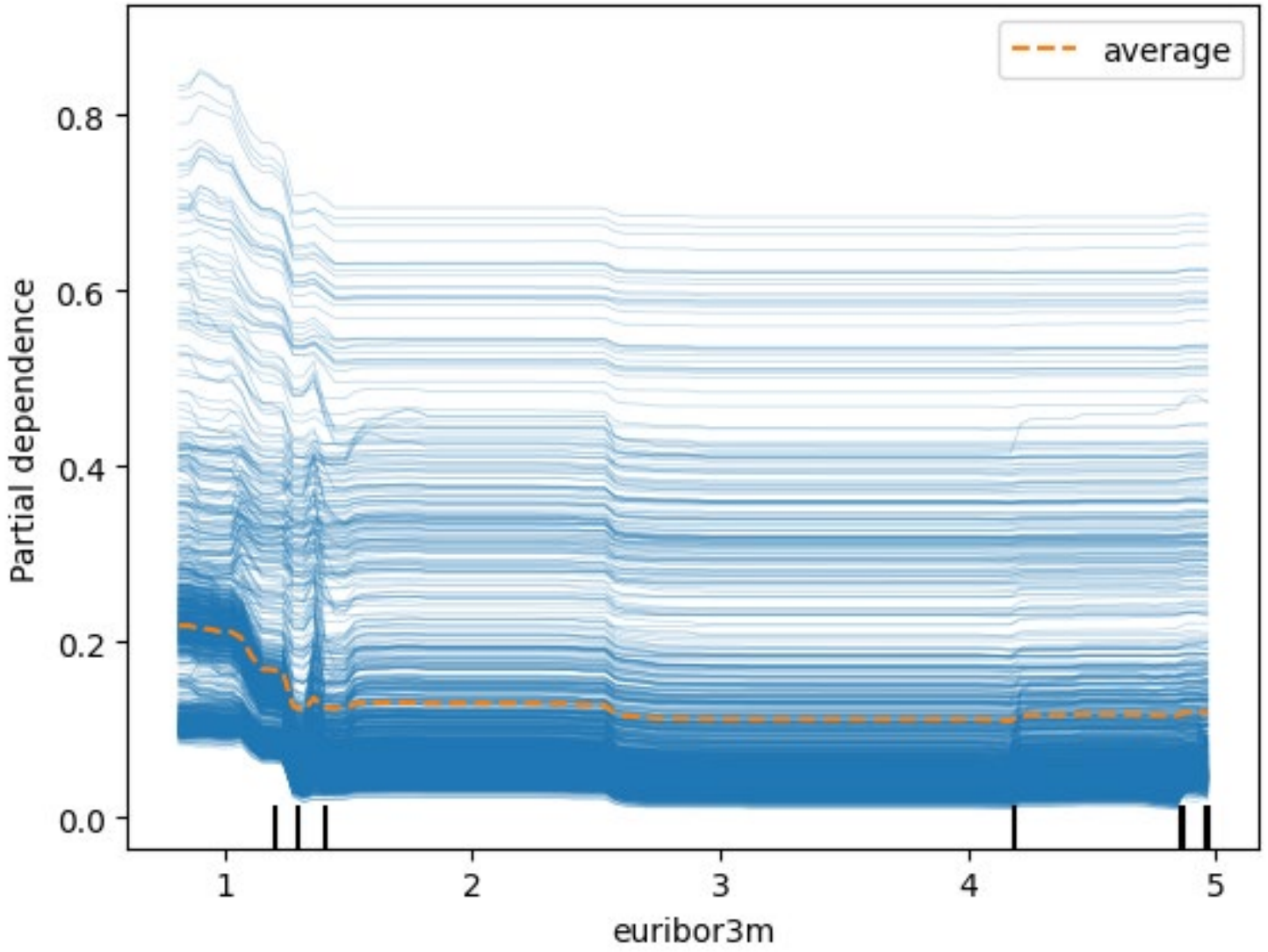
Bank Example: Partial Dependence Plots

Let's see how this works for the variable `euribor3m`

Each line is calculated on the same grid of values for `euribor3m`.

Each blue line corresponds to filling in the values for all X's *except* `euribor3m` with the values from a given observation.

The orange line is then the average of all the blue lines.



Code for Bank Deposit Example

Bank Example: Partial Dependence Plots

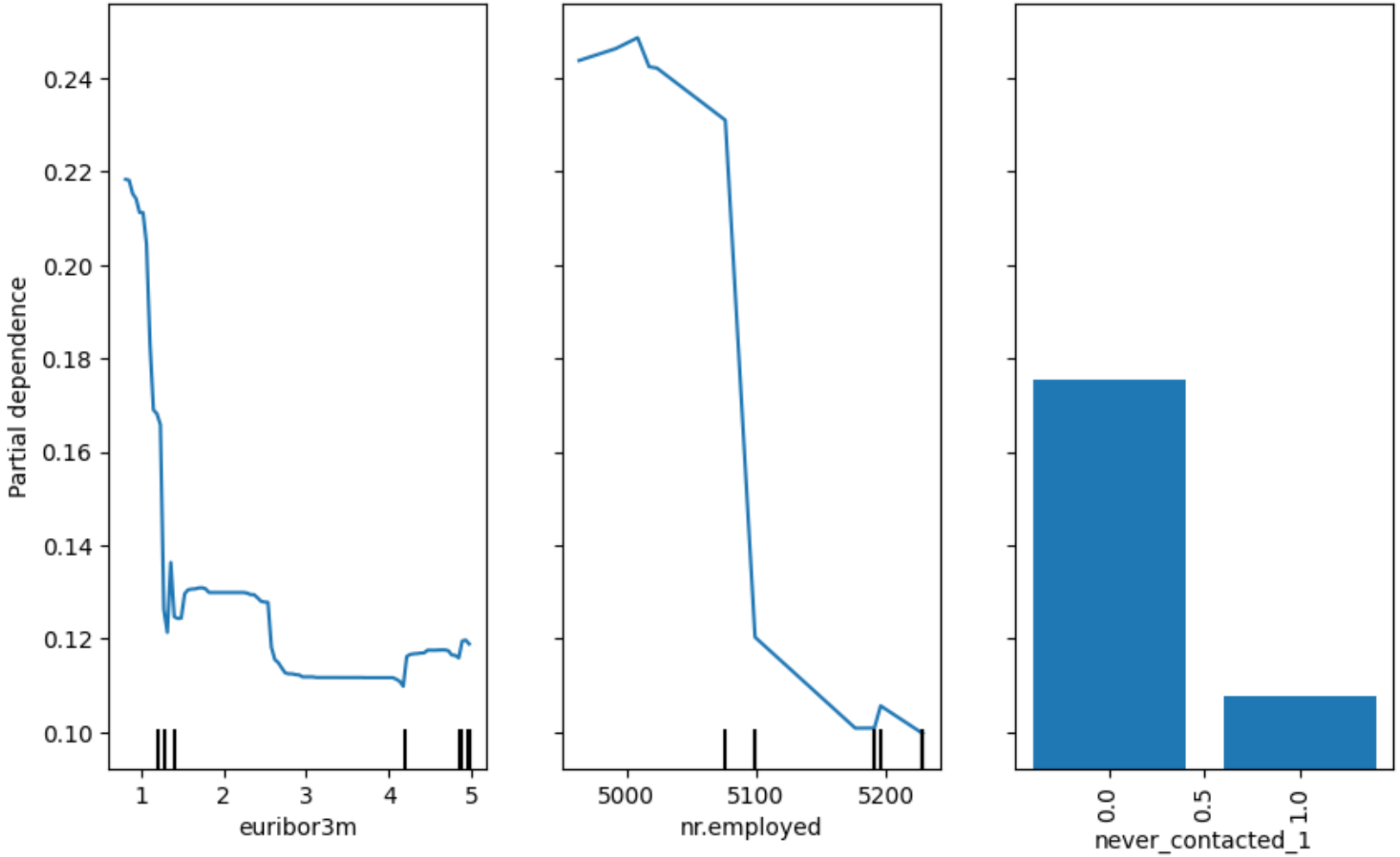
Partial dependence plots for three variables in bank example.

Some questions:

What does the y-axis mean?

What is different between never_contacted_1 and the other two variables?

What do these tell us about individual predictions?



SHAP

SHAP (SHapley Additive exPlanations):

- Based on a concept from game theory that tries to distribute credit among contributors in team settings

Basic idea is to decompose prediction into baseline and how each variable changes prediction away from baseline

- Baseline is *average value of prediction*
- SHAP are then (heuristically) the contribution of each variable to the overall prediction such that *baseline + sum(SHAP) = predicted value*
  - Calculated observation by observation

Advantages:

- Robust
- Can be applied to any learner
- Accounts for interactions

Disadvantages:

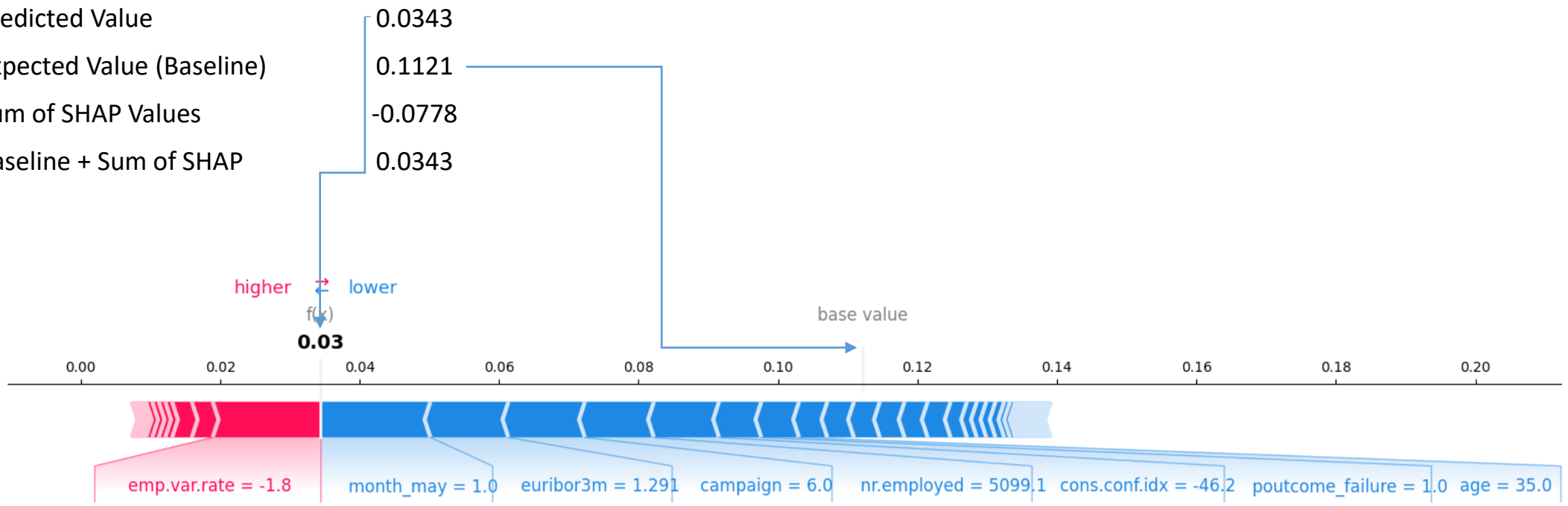
- Computation

Code for Bank Deposit Example

SHAP in bank example

Let's look at SHAP for one observation:

Predicted Value	0.0343
Expected Value (Baseline)	0.1121
Sum of SHAP Values	-0.0778
Baseline + Sum of SHAP	0.0343



Each "block" illustrates the "contribution" of one variable to this prediction.

Variable names only provided for most important variables

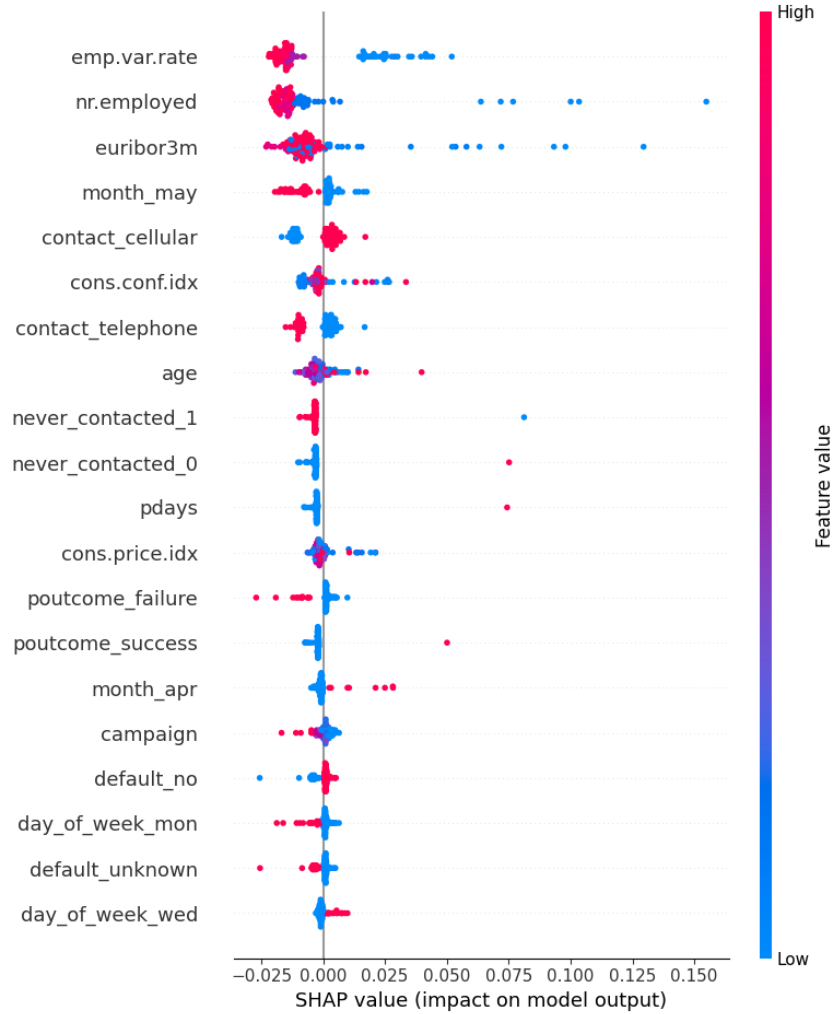
Code for Bank Deposit Example

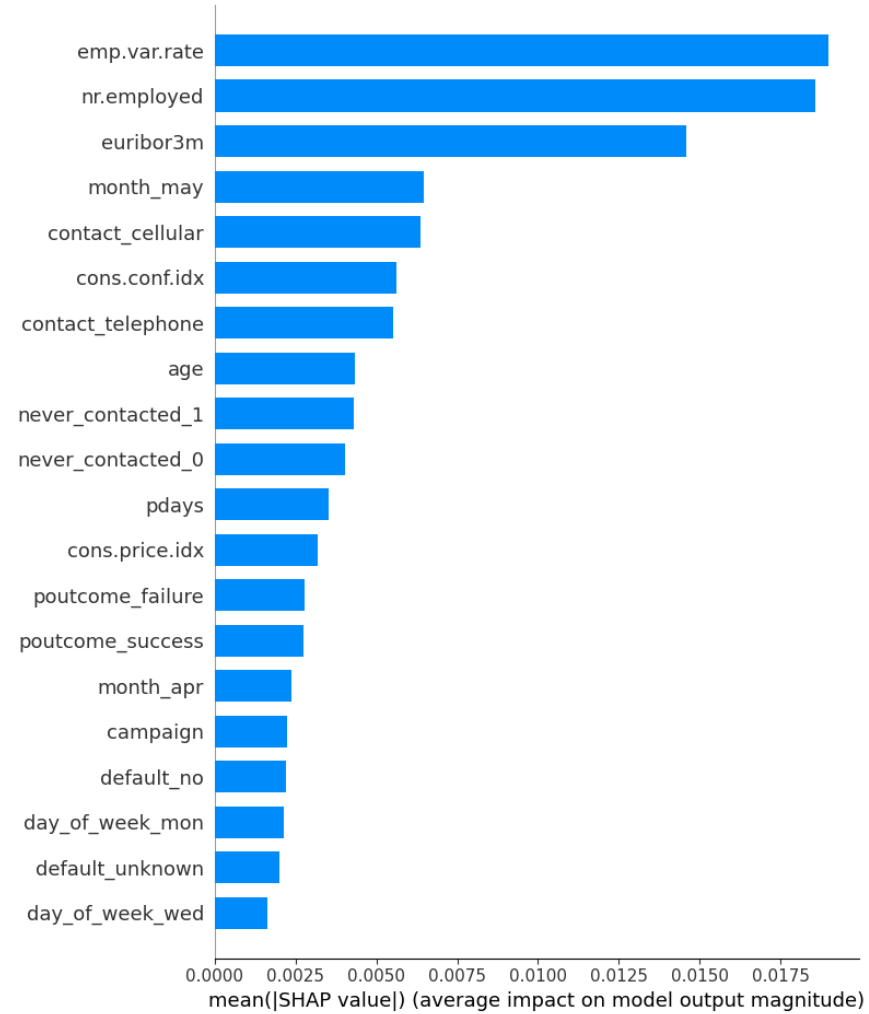
SHAP in bank example

Here we provide two common summary plots of SHAP values

Left plot (with the dots): Each dot represents a single observation. X-axis is SHAP value. Color of dot is value of input variable.

Right plot (bars): SHAP-based variable importance





Things to Remember

Knowing variables that are important in prediction rules tells you very little about mechanisms

- "Correlation does not imply causation"
  - (and lack of correlation does not imply lack of causation)

Think about correlated variables

Think about ultimate goal

Magnitudes of importance measures can be hard to think about

Most measures are variable by variable – can make variables that matter through *interactions* seem less important

Common interpretation devices are high-level summaries – can miss nuance and impact in specific use cases

4. Stacking and AutoML

Model Choice

Hard (impossible?) to know the right model to try in any given setting

- Already seen this in choosing tuning parameters via (cross-)validation

Should try several models and see which does best

- BUT, different models may perform better/worse for different instances

Stacking = Rather than choose one model, *combine models* to leverage strengths of each!

AutoML = Use machines to automate the whole model building process

- Try different learning approaches (e.g. random forests, boosting, neural networks …)
- Try different configurations of tuning parameters
- Combine models

Stacking

Stacking is a simple model combination idea.

Suppose you have $\mathrm{m} = 1, \ldots, M$ "base" models for predicting outcome Y

- Models trained using cross-validation
  - Let $\hat{Y}_i^m$ be the (out-of-sample/cross-validation) prediction for observation i based on model m
- Stacking takes $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to use as predictors for building a *meta*-model for predicting Y
  - For regression tasks, common implementation is to use $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to predict Y using linear regression
    - Can also used penalized linear regression – especially useful if m is large
  - For classification tasks, common implementation is to use $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to predict Y using (multinomial) logistic regression
    - Can also used penalized (multinomial) logistic regression – especially useful if m is large
  - I.e. we just treat the (out-of-sample) predictions produced by our different candidates as data that can be used to predict the outcome

Code for AutoML in Bank Example

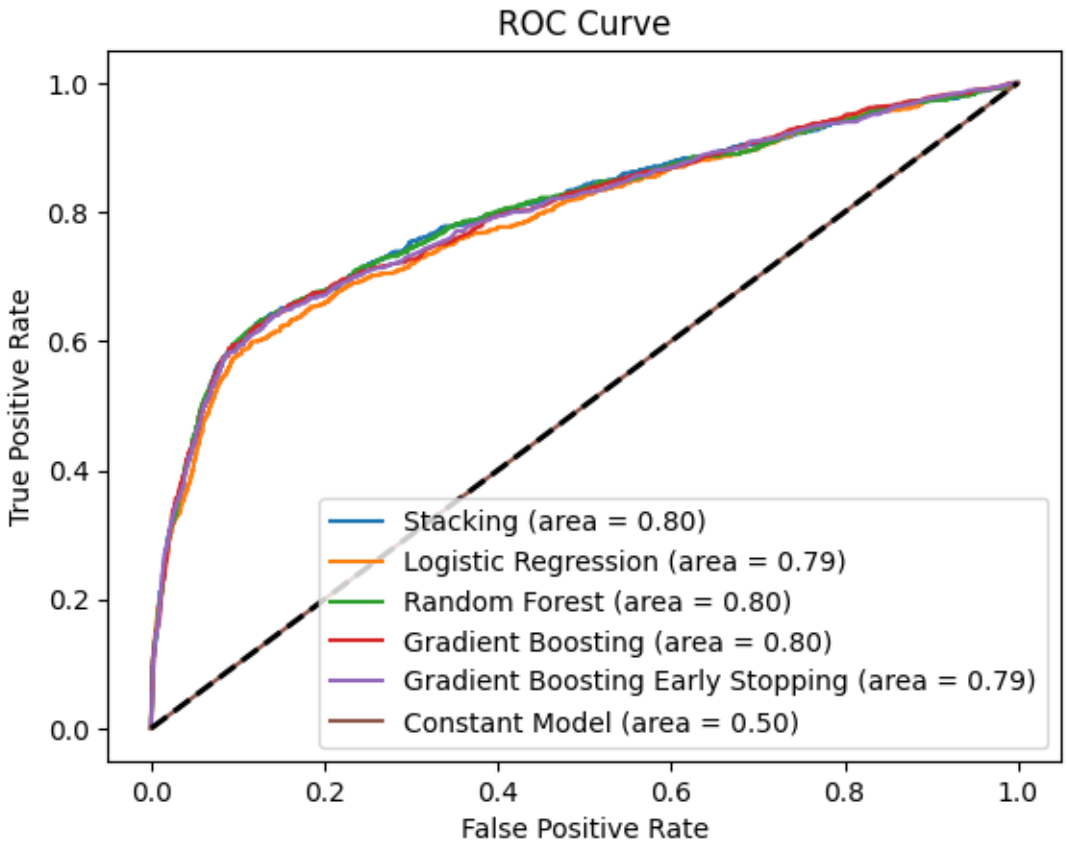
Stacking in Bank Example

Let's look at stacking in the bank deposit example using the learners from the previous sections.

- Combine the learners using logistic regression

Model	Logistic Regression Coefficient
Logistic	-0.222
Random Forest	5.006
Boosting	0.819
Boosting (Early Stopping)	1.031
Constant	-0.230

Stacking accuracy: 0.900



Code for AutoML in Bank Example

AutoML

The basic idea of automatic machine learning (AutoML) is straightforward:

- We're already using a computer to try lots of models
- Wrap the whole thing in a package that just automates going through models
- Ideally, a user would just indicate the target variable and predictor variables and the machine would do the rest
  - Not quite there in my experience

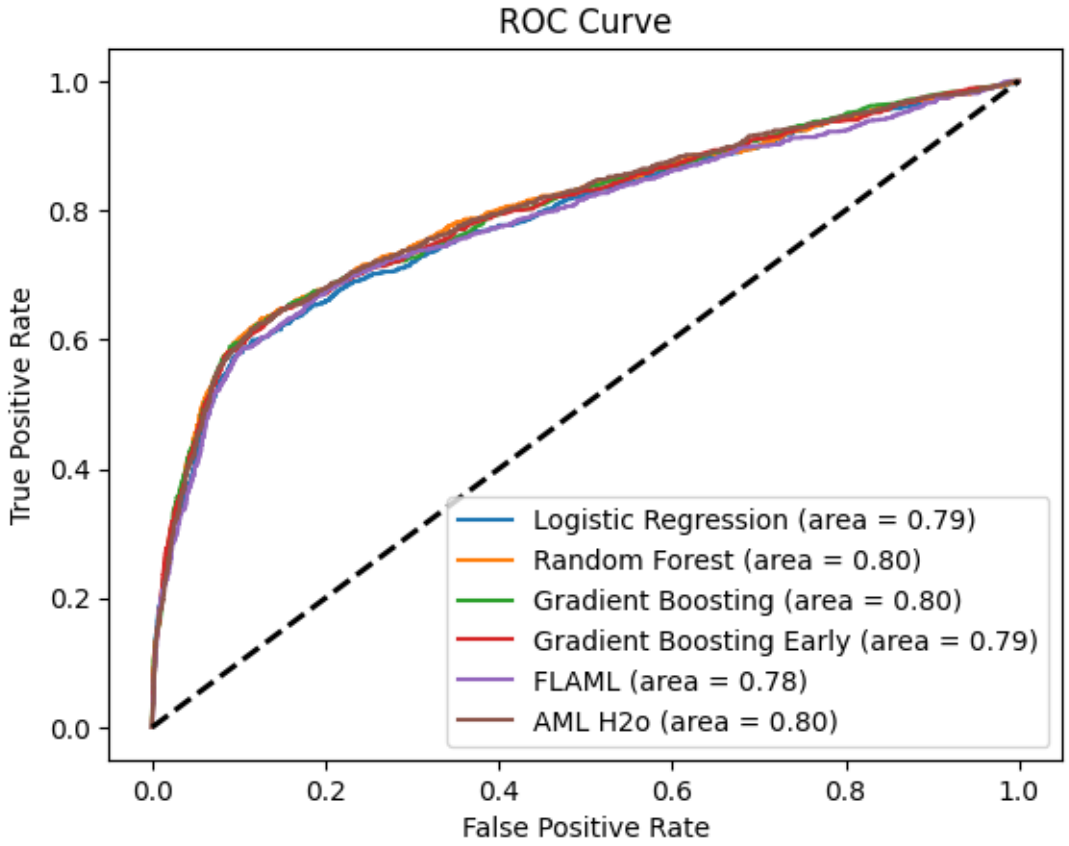
We've added two AutoML procedures in our bank example.

FLAML makes use of a time budget (set to 300s here).

- Accuracy = 0.896

H2o makes use of a set number for models (set here to 20).

- Accuracy = 0.891
- Predicts many more "1"s than other approaches



5. Summary

Summary

- "Classification" is jargon for supervised learning with a discrete outcome
  - Everything from Notes 1 still applies but people often use different evaluation metrics
- Random forests and boosted trees are simple examples of "black box" learners (that can also be used for regression)
  - Often very useful and competitive in settings with limited data
- Stacking is a simple model combination idea (that can also be used for regression)
- We're missing one important class of popular learners – Deep Neural Networks.
  - We'll give them their own treatment later!